WEB VULNERABILITY SCANNER & SECURITY DASHBOARD

Project Report

Project Type	Cybersecurity / Web Application Security
Technology Stack	Python, Flask, SQLite, HTML, CSS, JavaScript
Application	Local educational security assessment dashboard
Made By	Majithiya Jal Anandbhai

Abstract

The Web Vulnerability Scanner & Security Dashboard is a lightweight educational web application designed to perform a baseline review of a website's publicly observable security configuration. A user submits a URL and confirms authorization. The scanner makes a limited set of HTTP requests, checks selected security headers and cookie attributes, observes HTTPS/TLS behavior, looks for simple mixed-content indicators, and checks a small set of conventional public files. Findings are stored in an SQLite database and displayed in a dashboard with severity summaries, a heuristic score, scan history, and JSON/CSV report exports.

The project is intended for learning and demonstration. It is not a complete penetration-testing solution and cannot establish that a site is secure. Findings should be validated by a qualified tester and used only for systems that the operator owns or is explicitly authorized to assess.

Table of Contents

• 1. Introduction
• 2. Problem Statement
• 3. Objectives and Scope
• 4. System Requirements
• 5. Technology Stack
• 6. System Architecture
• 7. Modules and Features
• 8. Methodology and Workflow
• 9. Database Design
• 10. Security and Ethical Considerations
• 11. Testing Plan
• 12. Limitations
• 13. Future Enhancements
• 14. Conclusion
• Appendix A: Setup and Execution

1. Introduction

Web applications are exposed to a range of configuration risks, including missing browser security headers, weak cookie attributes, and incorrect HTTPS deployment. Manual checks are useful but can be repetitive for beginners. This project provides a simple interface that runs a small set of repeatable baseline checks and presents the results in a central dashboard.

The application uses a Python backend for scanning and persistence, Flask for routing and HTML rendering, SQLite for scan history, and a responsive frontend built with HTML, CSS, and JavaScript.

2. Problem Statement

Students and small development teams may lack a simple, beginner-friendly way to review common website security configuration issues and track previous assessments. The project addresses this learning need by combining a controlled baseline scanner with readable findings, remediation suggestions, and downloadable reports.

3. Objectives and Scope

Objectives

- Provide a dashboard to initiate an authorized website baseline scan.
- Check selected HTTP security headers and HTTPS/TLS request behavior.
- Inspect observed Set-Cookie values for Secure, HttpOnly, and SameSite attributes.
- Flag a server banner and simple possible mixed-content indicators.
- Check a small fixed set of conventional public paths: robots.txt, security.txt, and sitemap.xml.
- Store scan results and present severity totals, a heuristic score, and scan history.
- Export individual reports in JSON and CSV formats.

In scope

- Single submitted URL per scan, without authentication or JavaScript execution.
- Low-impact GET requests with request timeouts and certificate verification.
- Local use by default at 127.0.0.1:5000.
- Educational reporting and remediation guidance.

Out of scope

- Exploitation, payload injection, password guessing, brute force, and destructive testing.
- Authenticated crawling, full-site crawling, JavaScript rendering, and comprehensive CVE detection.
- Multi-user access control, production deployment hardening, and a claim of compliance certification.

4. System Requirements

Category	Requirement
Operating system	Windows, macOS, or Linux
Runtime	Python 3.10 or newer recommended
Dependencies	Flask 3.x compatible release; Requests 2.x compatible release
Browser	Recent Chrome, Edge, Firefox, or equivalent
Network	Internet access to reach the authorized target and install packages
Storage	Local writable directory for scanner.db

5. Technology Stack

Technology	Purpose
Python	Implements scanning logic, scoring, report construction, and data handling.
Flask	Provides routes and renders dashboard, history, and report pages.
Requests	Performs bounded HTTP requests with timeouts and TLS certificate validation.
SQLite	Persists scan metadata, findings, checks, scores, and timestamps locally.
HTML/CSS	Builds the responsive dashboard, tables, report cards, and status indicators.
JavaScript	Adds small interface behaviors such as clearing the URL field and disabling the submit button while scanning.

6. System Architecture

The application follows a small layered design. The browser sends form requests to Flask routes. The route validates the user's authorization confirmation and URL format, calls the scanner module, and saves the generated report in SQLite. Template pages then display the stored result. Export routes serialize a saved report as JSON or CSV.

Browser UI Dashboard / History / Report
↓ HTTP form requests
Flask application (app.py) Routing · authorization confirmation · persistence · exports
↓ calls
Scanner module (scanner.py) HTTP headers · TLS behavior · cookie flags · public paths
↓ returns report

SQLite database (scanner.db)
Scan history and serialized findings

7. Modules and Features

7.1 Dashboard

Displays total saved scans, average heuristic score, accumulated high/critical findings, total findings, severity distribution, and recent scan activity.

7.2 URL validation and authorization confirmation

The scan form requires the user to confirm ownership or permission. The URL normalizer accepts HTTP/HTTPS schemes, adds HTTPS when no scheme is supplied, and rejects malformed URLs and embedded credentials. The checkbox is a procedural safeguard, not proof of authorization.

7.3 Scanner engine

The scanner sends a request to the submitted URL using Requests with a timeout and certificate verification. It examines response headers, records status information, evaluates HTTP-to-HTTPS redirection, inspects observed cookie attributes, and makes GET requests to three fixed public paths. It does not exploit or fuzz the target.

7.4 Findings and remediation

Each finding includes a severity label, title, observed detail, and recommended action. Severity is a prioritization aid only; it is not a validated risk rating and should be confirmed manually.

7.5 History and exports

Completed scans are stored in SQLite. History pages open a saved report by scan ID. Export endpoints provide JSON for machine-readable use and CSV for spreadsheet review.

8. Methodology and Workflow

- User opens the local dashboard and enters a target URL.
- User confirms the target is owned or explicitly authorized for testing.
- The backend normalizes the URL and performs a limited HTTP request with a timeout and TLS verification.
- The scanner checks selected headers, HTTPS behavior, cookie attributes, a server banner, possible mixed-content references, and fixed public paths.
- Findings and check statuses are compiled; a heuristic score is calculated.
- The report is stored in SQLite and the browser is redirected to the result page.
- The user reviews recommendations and optionally downloads JSON or CSV.

Scoring model

The current demonstration score starts at 100 and subtracts 30 points per Critical finding, 20 per High, 10 per Medium, and 4 per Low finding, with a lower bound of zero. Informational findings do not reduce the score. This is a simple project-specific heuristic, not a recognized security certification or a quantitative measure of exploitability.

9. Database Design

The database contains a single scans table. Findings, checks, and severity counts are serialized as JSON text fields to keep the starter implementation compact.

Field	Type / role
id	Integer primary key; unique scan identifier.
created_at	Text timestamp for when the report was saved.
url	Text URL used for the report.
score	Integer heuristic score from 0 to 100.
findings_json	JSON text array of finding title, severity, detail, and recommendation.
checks_json	JSON text array of check names, statuses, and details.
counts_json	JSON text object with severity totals.
status_code	Integer HTTP response code where available; may be null.

10. Security and Ethical Considerations

- Only scan systems for which you have explicit authorization. The confirmation checkbox is not a substitute for permission.
- The application binds to 127.0.0.1 by default so the development server is not intentionally exposed to the local network.
- Requests have timeouts and certificate validation remains enabled.
- The scanner uses a fixed list of public paths and does not send exploit payloads, brute-force credentials, or attempt destructive actions.
- Do not deploy this development application publicly without adding authentication, CSRF protection, server-side rate limits, stronger URL/IP restrictions, secure secret management, logging, and production hosting controls.
- A production-grade scanner should also defend against server-side request forgery (SSRF), including loopback, private, link-local, and cloud metadata destinations, DNS rebinding, and redirect-to-private-address behavior. This starter code is intended for local controlled use and should not be exposed as a public scanning service.

11. Testing Plan

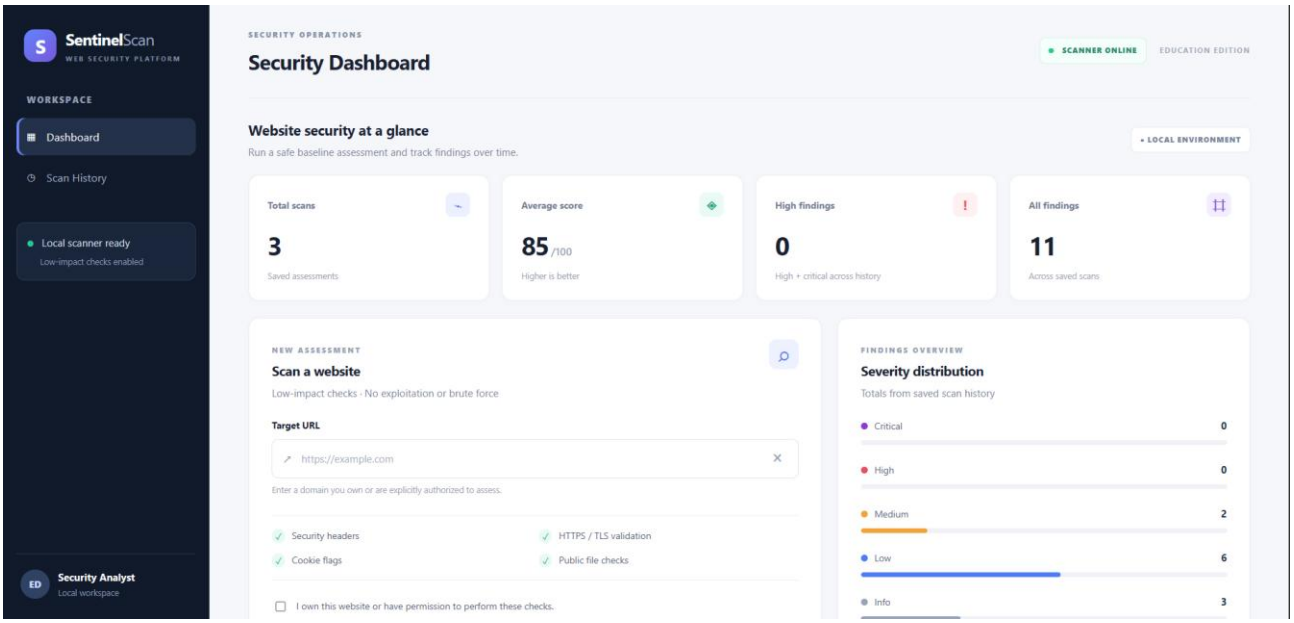
The following is a recommended test plan. It describes expected checks and is not a claim that every case has already been executed.

Test	Action	Expected result
Dashboard load	Open local root route.	Dashboard and empty-state UI render.
Missing authorization	Submit without checking consent.	Scan is rejected and a warning is shown.
Invalid URL	Enter malformed or unsupported URL.	Validation error is shown; no report is saved.
Authorized HTTPS site	Scan a controlled test site.	HTTP response, selected headers, and report are stored.

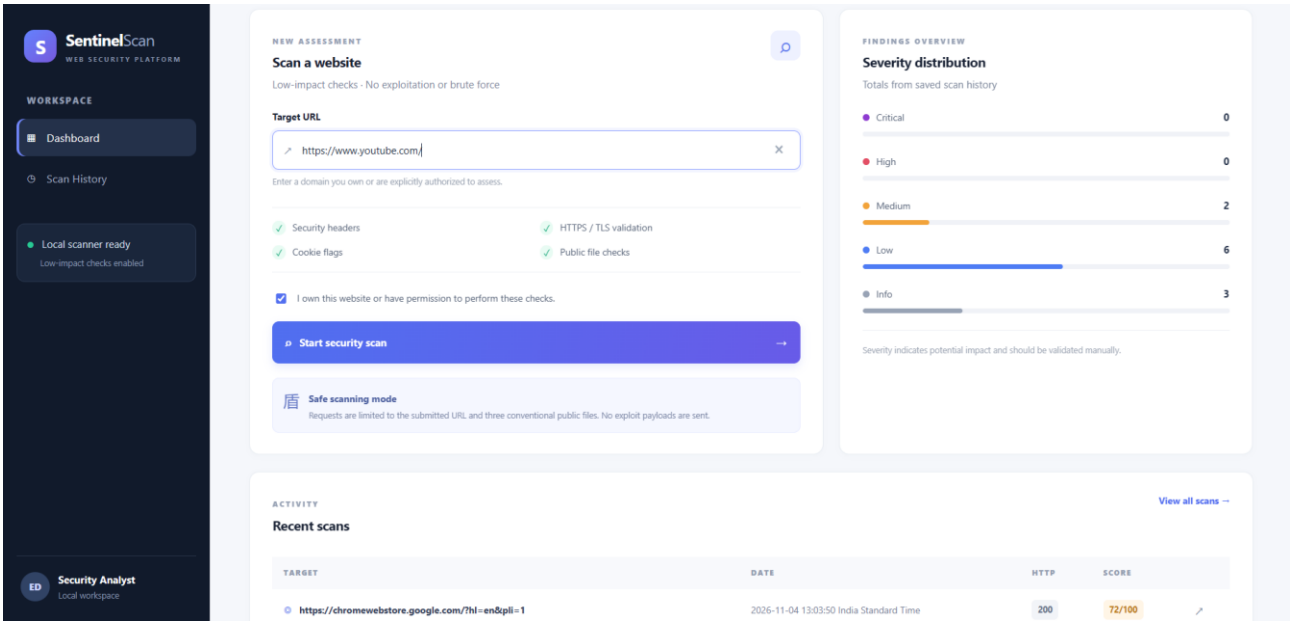
HTTP site with redirect	Use an authorized test endpoint that redirects to HTTPS.	Redirect behavior is recorded.
History	Complete a scan and revisit history.	Saved scan appears and opens correctly.
JSON / CSV export	Export a saved report in each format.	Download contains report and finding fields.
Test	**Action**	**Expected result**
Unreachable target	Use a controlled invalid/unreachable host.	A readable error is shown without crashing the app.

12. Implementation and Output Screenshots

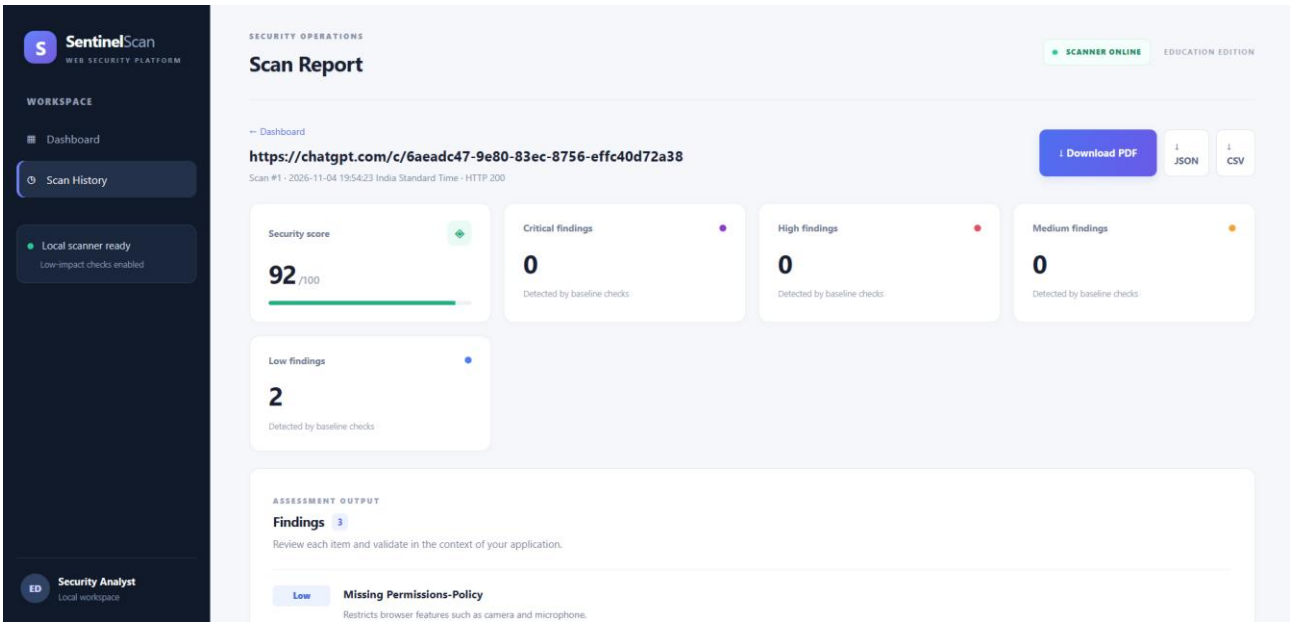
Main Dashboard



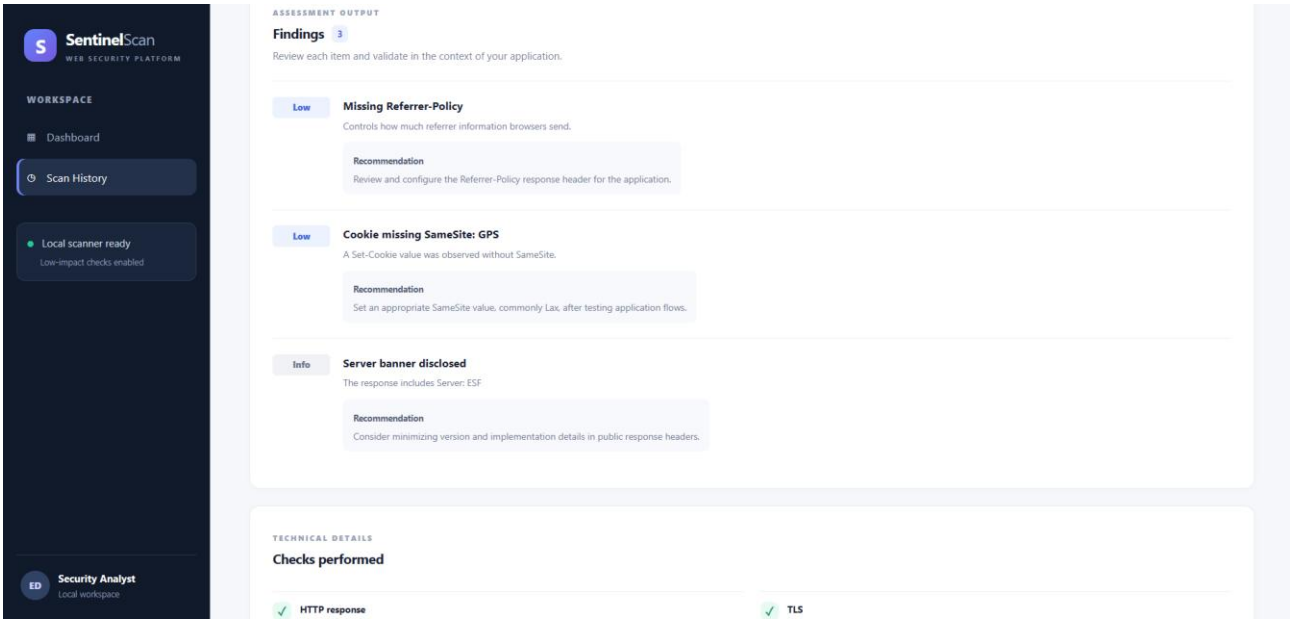
Website Scan Input



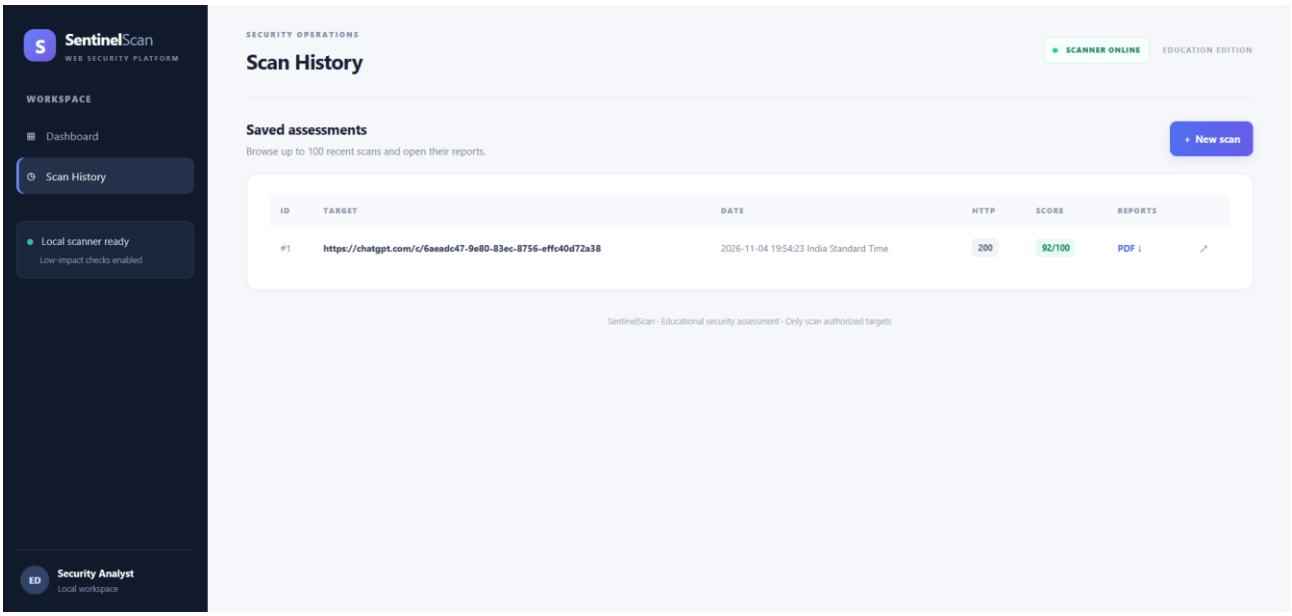
Scan Results



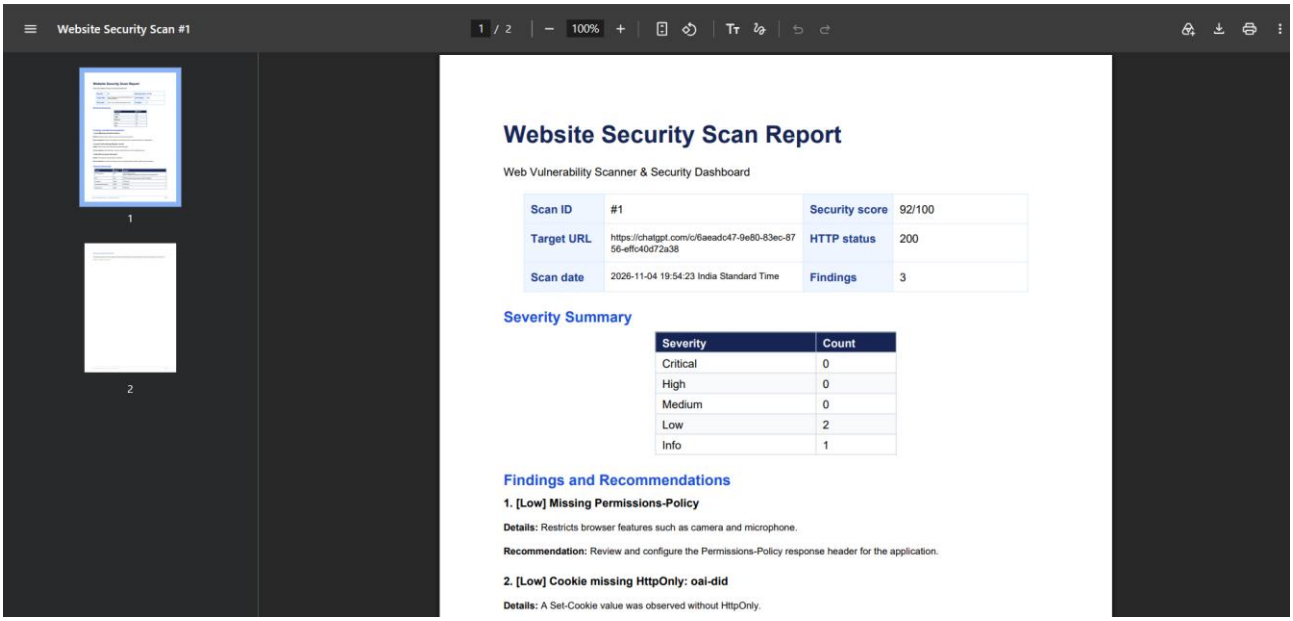
Detailed Findings



Scan History



Exported Report



13. Limitations

- Only the initial response and a small fixed set of public paths are inspected; JavaScript is not executed and links are not crawled.
- Checks are configuration heuristics and may produce false positives or miss issues.
- The score is an illustrative prioritization metric, not a recognized standard.
- Cookie analysis depends on the response exposing Set-Cookie headers to the client library.
- The application has no user accounts, role-based access, multi-tenant separation, or scan scheduling.
- The local demo does not provide a comprehensive SSRF defense and must not be made available as a public scanning endpoint.
- It does not replace a dedicated dynamic application security testing tool or professional penetration test.

14. Future Enhancements

- Add authentication, role-based access, CSRF protection, audit logs, and a hardened deployment model.
- Add safe scope allowlists and robust blocking of private/internal IP ranges and redirect destinations.
- Provide PDF reports, report comparison, trend charts, and configurable scan profiles.
- Add a controlled crawler with strict rate limits and an explicit allowlist.
- Integrate standards-based checks and link findings to OWASP guidance, with manual validation steps.
- Add unit tests, integration tests, and a dedicated test suite using local mock HTTP servers.
- Support background jobs and progress updates for longer authorized assessments.

15. Conclusion

The Web Vulnerability Scanner & Security Dashboard demonstrates how a lightweight Python web application can combine baseline security checks, findings presentation, persistent scan history, and report exports. It is suitable as a student project or starting point for learning web security automation. Its checks are intentionally limited and should be expanded and validated before any operational use. Authorization, scope control, and safe testing practices are essential whenever security tools interact with real websites.

Appendix A: Setup and Execution

A.1 Prerequisites

Install Python 3.10 or newer. Extract the supplied ZIP and open a terminal in the Web_Vulnerability_Scanner project directory.

A.2 Windows PowerShell

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python app.py
```

If script activation is blocked, run this in the same PowerShell window, then activate the environment:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

A.3 Open the application

Visit http://127.0.0.1:5000 in your browser. Enter a website you own or are authorized to test, check the authorization box, and click Start security scan. Completed reports are stored in scanner.db in the project directory.

A.4 Project file structure

```
Web_Vulnerability_Scanner/
■■■ app.py
■■■ scanner.py
■■■ requirements.txt
■■■ README.md
■■■ templates/
■  ■■■ base.html
■  ■■■ index.html
■  ■■■ results.html
■  ■■■ history.html
■■■ static/
■■■ css/style.css
■■■ js/dashboard.js
```